DBI Mitschrift

06.11.2026 – Embedding vs. Referencing

1. Embedding vs. Referencing

1.1. Beispiel: Social-Media-Post (Embedding)

Die Kommentare werden direkt im Post-Dokument **eingebettet**:

```
db.socialmediapost.insertOne({
  _id: "BN_061126",
  title: "ich mag sushi",
  text: "…",
  comments: [
    { _id: …, user: …, text: "…" },
    { _id: …, user: …, text: "…" }
  ]
})
```

Problem: Wenn der Post ultra viral geht, wächst das `comments`-Array immer weiter – das **Limit von 16 MB pro Dokument** kann erreicht werden.

1.2. Lösung: Referencing

Deswegen werden die Kommentare in eine **eigene Collection** ausgelagert und verweisen auf den Post:

```
db.comments.insertOne({
  _id: …,
  post_id: "BN_061126",   // Referenz auf den Post
  user: …,
  text: "…"
})
```

Zusammengeführt wird beim Lesen mit einem **Lookup** (`$lookup`) – das entspricht einem Join in SQL.

Ergänzung: Beispiel für einen Lookup:

```
db.socialmediapost.aggregate([
  { $lookup: {
      from: "comments",
      localField: "_id",
      foreignField: "post_id",
      as: "comments"
  } }
])
```

Prüfungsrelevant:

- Im Gegensatz zur relationalen DB kann man eine **Referenz angeben, die nicht existiert** – es gibt keine Fremdschlüssel-Constraints bzw. referentielle Integrität.
- Wann **Embedding** (Daten werden gemeinsam gelesen, begrenzte Menge) und wann **Referencing** (unbegrenzt wachsende Daten, z.B. Kommentare ⟶ 16-MB-Limit)?